

GAME PROGRAMMING COURSE — DESIGN PATTERN

シングルトン

Singleton — 便利さと危険性を正しく知る（Unity 公式 E-Book より）

対象：クラスまで既習の選抜クラス

SINGLE
TON

Unity公式 E-Book をベースに進めます

『ゲームプログラミングのパターンを活用してコードをレベルアップさせる』

Unity Technologies 公式 E-Book（日本語版）

Singleton の章: p.53～60

[unity3d.jp > game-ebooks](https://unity3d.jp/game-ebooks)

デザインパターンとは

- よく出会う問題に対する、先人たちの「**解き方の型**」

注意

- E-Book は **2021 LTS** 向け。Unity 6 では一部の命令名が変わっています（後で説明）

PART 1

シングルトンの作り方

目標: E-Book のコードを読み、仕組みを説明できるようになる

こんな場面、どう書きますか？

敵を倒したら
スコア +100

スコアを管理する
GameManager

倒したときに
効果音を鳴らす

音を管理する
AudioManager

画面の
スコア表示を更新

表示を担当する
ScoreUI

問い

- Enemy は、シーンのどこかにいる **GameManager** や **AudioManager** を どうやって見つけて使えばいい？

悪い例①

毎回シーン全体から探す

C# Enemy.cs

```
1 public class Enemy : MonoBehaviour
2 {
3     private void OnDefeated()
4     {
5         // 倒されるたびにシーン全体から探す
6         GameManager gm = FindAnyObjectByType<GameManager>();
7         gm.AddScore(100);
8
9         AudioManager am = FindAnyObjectByType<AudioManager>();
10        am.PlaySE("defeat");
11    }
12 }
```

問題点

- 探す処理は重い（全オブジェクトを調べる）
- 置き忘れると **null** でエラー
- うっかり**2つ**置いても気づけない

Inspector で全部の敵にドラッグする

C# Enemy.cs

```
1 public class Enemy : MonoBehaviour
2 {
3     [SerializeField] private GameManager gameManager;
4     [SerializeField] private AudioManager audioManager;
5
6     private void OnDefeated()
7     {
8         gameManager.AddScore(100);
9         audioManager.PlaySE("defeat");
10    }
11 }
```

問題点

- プレハブはシーン内のオブジェクトを参照できない
- Instantiate した敵は **None** のまま
- Spawner から渡す...とバケツリレーが始まる

static = クラスにひとつだけの変数

C# static の確認

```

1  public class Counter
2  {
3      public static int total;    // クラスに1つだけ
4      public int mine;          // インスタンスごと
5  }
6
7  Counter a = new Counter();
8  Counter b = new Counter();
9  a.mine = 1;
10 b.mine = 2;
11 Counter.total = 10;    // クラス名.変数名
    
```

static のポイント

- インスタンスではなくクラスに属する
- new しなくても クラス名.変数名 で使える
- プログラム全体で**1つ**を共有

シングルトンパターンとは

- インスタンスが 1 つしかないことを保証する
- その 1 つにどこからでもアクセスできるようにする
- 向いているもの：ゲーム全体の流れを管理するマネージャー、オーディオ、ファイル保存など

POINT

①は「重複を消す仕組み」、②は **public static** で実現する。後半の危険は、ほぼ全部が②「どこからでも」から生まれる。

SimpleSingleton (E-Book p.55)

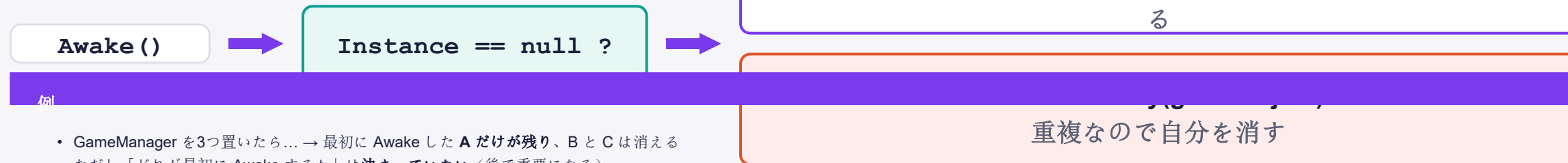
C# SimpleSingleton.cs

```
1 using UnityEngine;
2
3 public class SimpleSingleton : MonoBehaviour
4 {
5     public static SimpleSingleton Instance;
6
7     private void Awake()
8     {
9         if (Instance == null)
10         {
11             Instance = this;
12         }
13         else
14         {
15             Destroy(gameObject);
16         }
17     }
18 }
```

ポイント

- **public static Instance** → どこからでも SimpleSingleton.Instance
- **Awake** で「自分が最初か？」を判定
- 2個目以降は自分を破棄

Awake で何が起こっているか



GameManager をシングルトンにすると

C# GameManager.cs

```

1 public class GameManager : MonoBehaviour
2 {
3     public static GameManager Instance;
4     private int score;
5
6     private void Awake()
7     {
8         // SimpleSingleton と同じ処理
9     }
10
11     public void AddScore(int value)
12     {
13         score += value;
14     }
15 }

```

C# Enemy.cs

```

1 // どこからでも1行で呼べる
2 GameManager.Instance.AddScore(100);

```

悪い例①②が解決

- 探さない (Find 不要)
- ドラッグしない (Inspector 不要)
- プレハブから生成した敵でも使える

このままだと困ることが2つ

① シーンを切り替えると消える

SceneManager.LoadScene で

前のシーンのオブジェクトは全部破棄される。

スコアも一緒に消える。

解決 → DontDestroyOnLoad

シーンを切り替えても消えないようにする

② 先に **Hierarchy** に置く必要がある

置き忘れたシーンでは

Instance が **null** のまま。

テスト用シーンを作るたびに置く手間。

解決 → 遅延初期化

最初に使われた瞬間に、なければ自動で作る

DontDestroyOnLoad – シーンをまたいで生き残る

C# Program.cs

```
1 DontDestroyOnLoad(gameObject);
```

ポイント

- 呼ぶと、そのオブジェクトは「**DontDestroyOnLoad**」という特別なシーンへ移動
- シーンを読み込み直しても破棄されない
- ルート（親を持たない）オブジェクトにしか効かない

再生中の Hierarchy

▼ **GameScene**

Player / Enemy / Canvas ...

▼ **DontDestroyOnLoad**

GameManager

遅延初期化 — 使われた瞬間に用意する

C# プロパティの get

```
1 private static Singleton instance;
2
3 public static Singleton Instance
4 {
5     get
6     {
7         if (instance == null)
8         {
9             SetupInstance(); // なければ用意する
10        }
11        return instance;
12    }
13 }
```

ポイント

- 外からは変数のように **Singleton.Instance** と読める
- 中身はメソッド: 読まれるたびに **get** が動く
- なければ作ってから返す → 置き忘れても動く

Instance と Awake

C# Singleton.cs (前半)

```
1 public class Singleton : MonoBehaviour
2 {
3     private static Singleton instance;
4     public static Singleton Instance
5     {
6         get
7         {
8             if (instance == null) { SetupInstance(); }
9             return instance;
10        }
11    }
12    private void Awake()
13    {
14        if (instance == null)
15        {
16            instance = this;
17            DontDestroyOnLoad(this.gameObject);
18        }
19        else { Destroy(gameObject); }
20    }
```

変わったところ

- get: なければ **SetupInstance**
- Awake: 最初の1つなら **DontDestroyOnLoad**
- 2つ目以降は破棄

SetupInstance – 探して、なければ作る

C# Singleton.cs (後半)

```
1 private static void SetupInstance()
2 {
3     instance = FindAnyObjectByType<Singleton>();
4     if (instance == null)
5     {
6         GameObject gameObj = new GameObject();
7         gameObj.name = "Singleton";
8         instance = gameObj.AddComponent<Singleton>();
9         DontDestroyOnLoad(gameObj);
10    }
11 }
12 }
```

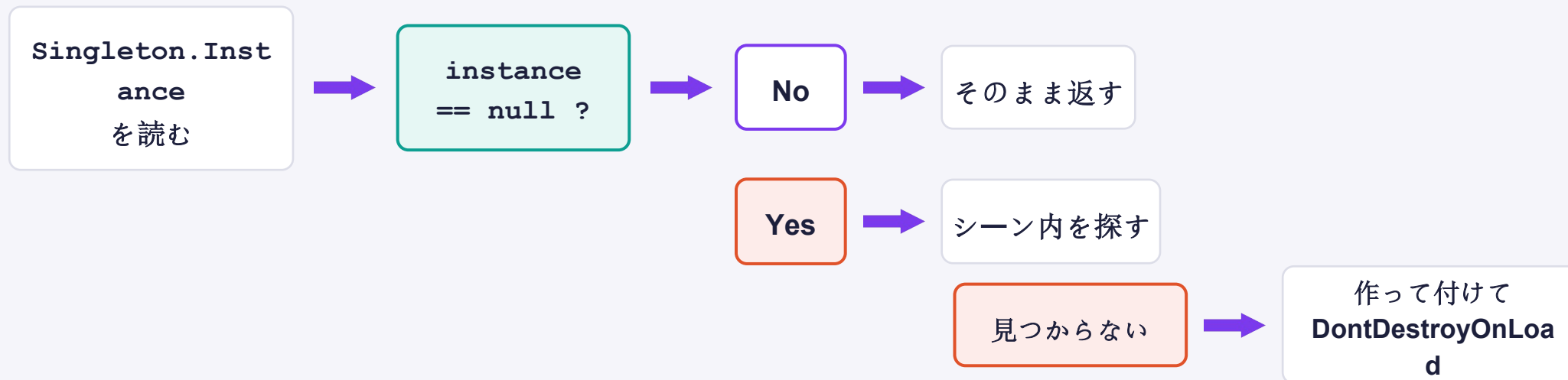
流れ

- シーン内を探す
- なければ **GameObject** を新しく作り、**AddComponent** で自分を付ける

Unity 6 での注意

- E-Book は **FindObjectOfType** → Unity 6 では非推奨（警告が出る）
- 代わりに **FindAnyObjectByType**

誰かが Instance を読んだとき



どこから、いつ読まれても「必ず1つ存在する」状態を作るのがねらい

マネージャーが増えたらどうする？

GameManager

Singleton のコード 30行
+ スコアの処理

AudioManager

Singleton のコード 30行
+ 音の処理

SaveManager

Singleton のコード 30行
+ 保存の処理

考え方

- 同じ30行を**3回**コピペ → バグを直すときも**3か所**
- 違うのはクラスの**型**だけ → 型を「あとから入れられる」ようにすればよい = ジェネリック

<T> は「型を入れる穴」

C# 実はもう使っている

```
1 List<int> numbers;  
2 List<string> names;  
3 GetComponent<Rigidbody2D>();
```

C# 自分で作るとこうなる

```
1 public class Box<T>  
2 {  
3     public T item;  
4 }  
5  
6 Box<int> a = new Box<int>(); // item は int  
7 Box<string> b = new Box<string>(); // item は string
```

ポイント

- **T** は使うときに決まる型
- **where T : Component** → T に入れられるのは Component の仲間だけ
- Singleton<T> は「T を1つだけにする」部品

Singleton<T> 前半

C# Singleton.cs (前半)

```
1 public class Singleton<T> : MonoBehaviour where T : Component
2 {
3     private static T instance;
4
5     public static T Instance
6     {
7         get
8         {
9             if (instance == null)
10            {
11                instance = FindAnyObjectByType<T>();
12                if (instance == null) { SetupInstance(); }
13            }
14            return instance;
15        }
16    }
17
18    public virtual void Awake()
19    {
20        RemoveDuplicates();
21    }
```

ポイント

- Singleton → **T** に置き換わっただけ
- Awake は **virtual** → 子クラスで上書きできる

Singleton<T> 後半

C# Singleton.cs (後半)

```
1     private static void SetupInstance()
2     {
3         instance = FindAnyObjectByType<T>();
4         if (instance == null)
5         {
6             GameObject gameObj = new GameObject();
7             gameObj.name = typeof(T).Name;
8             instance = gameObj.AddComponent<T>();
9             DontDestroyOnLoad(gameObj);
10        }
11    }
12    private void RemoveDuplicates()
13    {
14        if (instance == null)
15        {
16            instance = this as T;
17            DontDestroyOnLoad(gameObject);
18        }
19        else { Destroy(gameObject); }
20    }
21 }
```

ポイント

- **typeof(T).Name** → "GameManager" などの名前になる
- **this as T** → 自分を T 型として扱う

継承するだけでシングルトンになる

C# Program.cs

```
1 public class GameManager : Singleton<GameManager>
2 {
3     private int score;
4     public void AddScore(int value) { score += value; }
5 }
6
7 public class AudioManager : Singleton<AudioManager>
8 {
9     public void PlaySE(string name) { /* 音を鳴らす */ }
10 }
11
12 // 使う側
13 GameManager.Instance.AddScore(100);
14 AudioManager.Instance.PlaySE("defeat");
```

継承の注意

- 子クラスで Awake を書くなら **override** して **base.Awake()** を必ず呼ぶ
- 忘れると重複が消えない

PART 2

シングルトンの危険性

目標：便利さの裏にある危険を、具体的に説明できるようになる

なぜ嫌われるのか

「アンチパターン」と呼ばれることもある

E-Book より

- E-Book でも「最も評判が悪いデザインパターンの1つ」と紹介されている
- 理由: 使い勝手が良すぎて乱用される
- 不要なグローバル状態と依存関係が生まれる

シングルトンの正体

クラスの形をした グローバル変数

どこからでも読める = どこからでも書ける
= どこからでも壊せる

危険①

どこからでも書き換えられる

C# Program.cs

```
1 public class GameManager : MonoBehaviour
2 {
3     public static GameManager Instance;
4     public int score;    // 誰でも書き換え可能
5 }
6
7 // Enemy.cs
8 GameManager.Instance.score += 100;
9 // Item.cs
10 GameManager.Instance.score += 50;
11 // Trap.cs
12 GameManager.Instance.score -= 30;
13 // DebugCheat.cs
14 GameManager.Instance.score = 99999;
15 // ResultUI.cs (表示用のつもりが...)
16 GameManager.Instance.score = 0;
```

起きること

- スコアが急に 0 になった。犯人はどこ？
- 書き換える場所がプロジェクト中に散らばる
- 検索して全部調べるしかない

依存関係が隠れる

C# Player.cs

```
1 public class Player : MonoBehaviour
2 {
3     [SerializeField] private float speed;    // Inspectorで見える
4
5     private void Update()
6     {
7         // ↓ 外からは見えない依存が5つ
8         if (GameManager.Instance.IsPaused) return;
9         InputManager.Instance.Read();
10        AudioManager.Instance.PlaySE("step");
11        SaveManager.Instance.Mark();
12        UIManager.Instance.Refresh();
13    }
14 }
```

起きること

- Inspector にもクラス宣言にも出てこない
- Player だけテスト用シーンに置く → マネージャーが勝手に生成されたりエラーになる
- 別のゲームに持っていけない

危険③

密結合 — 1か所の変更が全体に広がる



周りの全員が GameManager を直接呼んでいる

起きること

- AddScore(int) を AddScore(int, Vector3) に変えたら？
- → 呼んでいる全ファイルがエラー
- 他のパターンは依存を切り離すためのもの。シングルトンは逆

実行順序の罠 – Awake の順番は決まっていない

C# ScoreUI.cs (SimpleSingleton 版)

```
1 public class ScoreUI : MonoBehaviour
2 {
3     private int startScore;
4
5     private void Awake()
6     {
7         // GameManager の Awake が
8         // まだ動いていなかったら...?
9         startScore = GameManager.Instance.score;
10    }
11 }
```

起きること

- Instance がまだ null →
NullReferenceException
- 別オブジェクト同士の **Awake** の順番は保証されない
- 「エディタでは動くのにビルドで落ちる」が起こりうる
- たまたま動いている状態が一番危険

危険⑤

参考資料のコードにも罠がある

1. ScoreUI の Awake が先に動き、GameManager.Instance を読む

2. instance が null なので Find で探す → まだ **Awake** 前の本物が見つかり代入される

3. 次に本物の GameManager の Awake → RemoveDuplicates()

4. instance は null ではない → 「重複だ」と判断して Destroy

結果

唯一の **GameManager** が
自分で自分を消す

しかも DontDestroyOnLoad も呼ばれていない

「自分以外なら重複」と判定する

C# Singleton.cs (修正版)

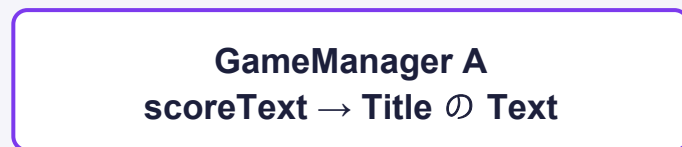
```
1 private void RemoveDuplicates()
2 {
3     if (instance == null)
4     {
5         instance = this as T;
6     }
7     else if (instance != this)    // 自分以外なら重複
8     {
9         Destroy(gameObject);
10        return;
11    }
12    DontDestroyOnLoad(gameObject);
13 }
```

修正のポイント

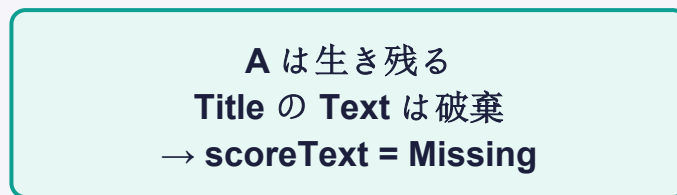
- instance が自分自身なら消さない
- 残る側は必ず **DontDestroyOnLoad**
- Destroy してもそのフレーム中は処理が続く → return で抜ける

DontDestroyOnLoad とシーン参照の罠

① Title シーン



② Game シーンへ



③ Title に戻る



注意

残った **A** が **Missing** を参照して **MissingReferenceException**。シーンをまたぐシングルトンにシーン内オブジェクトの参照を持たせない。

状態がいつまでも残り続ける

C# Program.cs

```
1 // ゲームオーバー → リトライ
2 SceneManager.LoadScene("Game");
3
4 // GameManager は DontDestroyOnLoad なので
5 // score も isBossDefeated も前回のまま!
```

起きること

- リトライしてもスコアやフラグが戻らない
- 対策: **ResetGame()** を用意し、いつ呼ぶかを決めておく
- エディタの Enter Play Mode 設定でドメインリロードを切ると、**static** が前回の再生の値のままだる

終了時に「幽霊」が生まれる

C# Enemy.cs

```
1 public class Enemy : MonoBehaviour
2 {
3     private void OnDestroy()
4     {
5         AudioManager.Instance.PlaySE("vanish");
6     }
7 }
```

起きること

- 再生停止・ゲーム終了時は破棄の順番が決まっていない
- AudioManager が先に消える → Instance を読む → 遅延初期化で新しく作られる
- 「Some objects were not cleaned up when closing the scene」エラー

危険⑨

「とりあえずシングルトン」の末路

GameManager

ScoreManager

PlayerManager

EnemyManager

BulletManager

UIManager

EffectManager

StageManager

TimeManager

↑ 全部どこからでも書き換え可能

起きること

- GameManager が**2000**行の「神クラス」に
- 2人プレイを作ろうとしたら PlayerManager.Instance が**1人前提**で総作り直し
- チーム開発で全員が同じファイルを触り衝突